

An Introduction to Microsoft Graph

Rob Windsor
rob@robwindsor.com
@robwindsor

About Me



Rob Windsor

.NET/Microsoft 365 developer, trainer, author
Microsoft MVP | Microsoft 365 Development | Microsoft Graph



Twitter: <https://twitter.com/robwindsor>

LinkedIn: <https://www.linkedin.com/in/rwindsor>

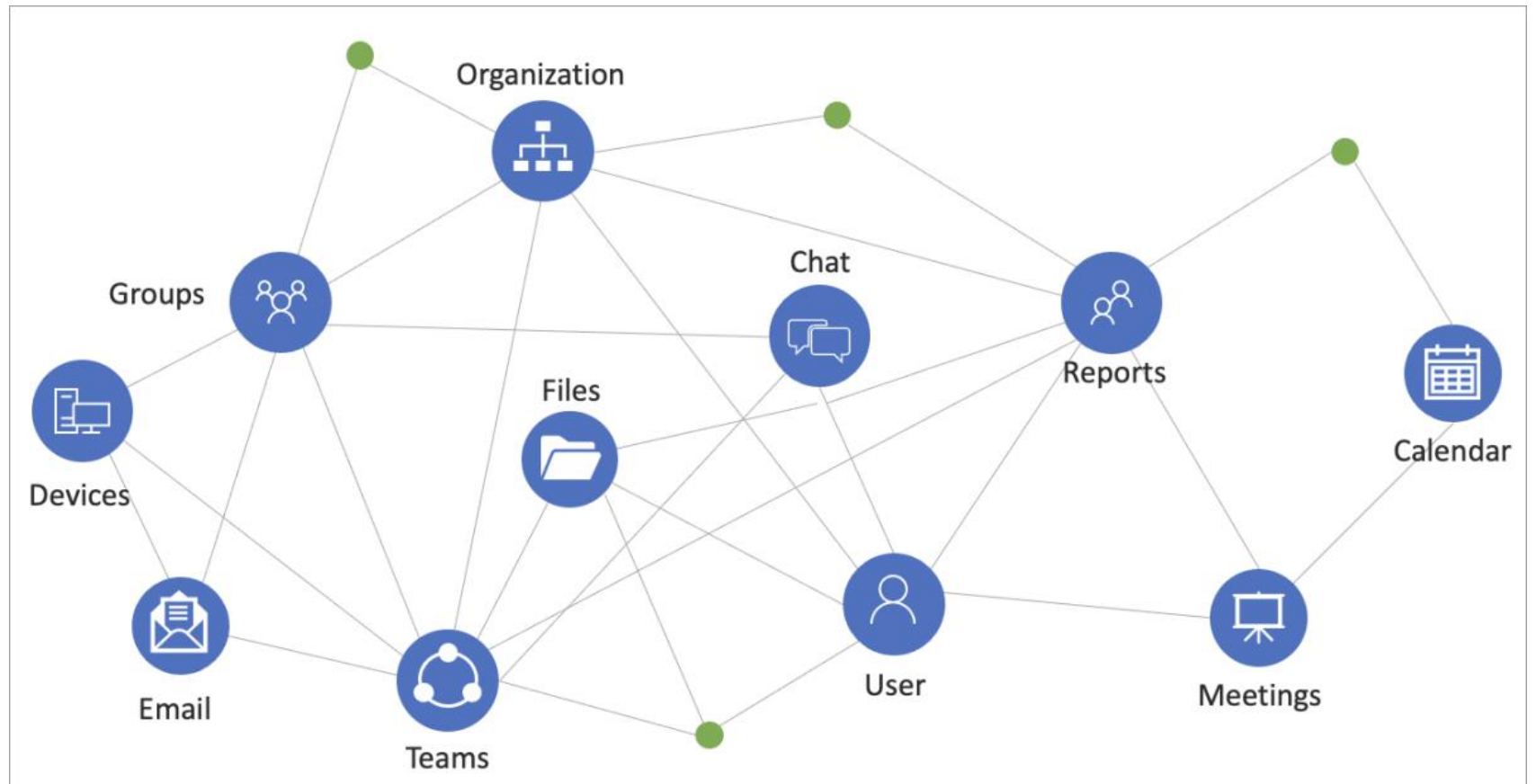
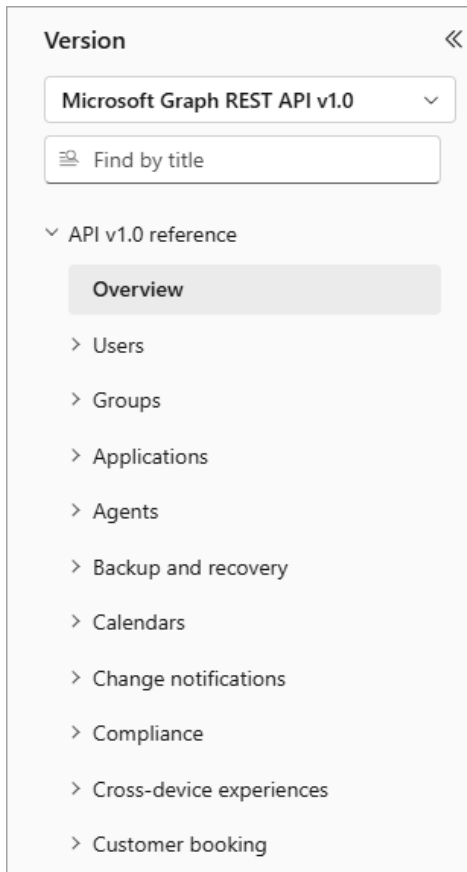
Blog: <https://robwindsor.hashnode.dev>

YouTube: <https://www.youtube.com/@RobWindsor>

GitHub: <https://github.com/rob-windsor>

Microsoft Graph

Microsoft Graph exposes Microsoft cloud data and capabilities through a common API



Microsoft Graph

Microsoft Graph uses REST and OData conventions:

- Standard HTTP methods: GET, POST, PATCH, PUT, DELETE
- Resources are addressed with URLs
- Request bodies and responses are typically JSON
- OData query options can shape results: \$select, \$filter, \$orderby, \$top, \$expand, \$count
- Collections may be paged using @odata.nextLink

Support for individual OData query options varies by endpoint.

Navigating Microsoft Graph

/me

resource

/me/messages

related collection

/me/drive

related resource

/me/drive/root/children

resource path through
relationships

Anatomy of a Microsoft Graph Request

```
GET https://graph.microsoft.com/v1.0/me/messages?$top=5
```

GET

HTTP method

v1.0

API version

me/messages

resource path

\$top=5

query

Authorization: Bearer ...

Microsoft Graph is a protected API. Applications must send an access token with the necessary permissions

Common Microsoft Graph Requests

Operation	URL
Get my profile	https://graph.microsoft.com/v1.0/me
Get my email messages	https://graph.microsoft.com/v1.0/me/messages
Get my OneDrive files	https://graph.microsoft.com/v1.0/me/drive/root/children
List users	https://graph.microsoft.com/v1.0/users
Get a specific user	https://graph.microsoft.com/v1.0/users('MeganB@robwindsortest980.onmicrosoft.com')
List email messages for a specific user	https://graph.microsoft.com/v1.0/users('MeganB@robwindsortest980.onmicrosoft.com')/messages
Get selected profile properties	https://graph.microsoft.com/v1.0/me?\$select=displayName,jobTitle,mail
Get five email messages	https://graph.microsoft.com/v1.0/me/messages?\$top=5
Get unread messages	https://graph.microsoft.com/v1.0/me/messages?\$filter=isRead eq false
List users by display name	https://graph.microsoft.com/v1.0/users?\$orderby=displayName

Graph Explorer

The screenshot displays the Microsoft Graph Explorer web application. On the left, a sidebar titled 'Sample queries' includes links to 'API Explorer' and 'History', a search bar, and a list of 340 search results categorized by topics like 'Getting Started', 'Agents', 'Applications', 'Batching', 'Compliance (beta)', 'Edge', 'Excel', 'Extensions', 'Groups', 'Identity and Access', 'Insights', 'Microsoft Teams', and 'Microsoft Teams (beta)'. The main area features a 'New to Graph Explorer?' banner, a request configuration bar with a 'GET' method, 'v1.0' version, and the URL 'https://graph.microsoft.com/v1.0/me', and a 'Run query' button. Below this, tabs for 'Request Body', 'Request Headers', 'Modify Permissions', and 'Access token' are visible. A green status bar indicates a successful 'OK - 200 - 715 ms' response. The 'Response preview' tab is active, showing a JSON response for the current user. A warning message at the top of the response area states: 'You are currently using a sample account. Sign in to access your own data.'

```
{
  "@odata.context": "https://graph.microsoft.com/v1.0/$metadata#users/$entity",
  "@microsoft.graph.tips": "This request only returns a subset of the resource's properties. Your app will need to use $select to return non-default properties. To find out what other properties are available for this resource see https://learn.microsoft.com/graph/api/resources/user",
  "businessPhones": [
    "+1 858 555 0110"
  ],
  "displayName": "Alex Wilber",
  "givenName": "Alex",
  "jobTitle": "Marketing Assistant",
  "mail": "AlexW@M365x64025835.OnMicrosoft.com",
  "mobilePhone": null,
  "officeLocation": "131/1104",
  "preferredLanguage": null
}
```

Use Graph Explorer to:

- Run requests
- Inspect responses
- Check permissions
- Generate code

Calling Microsoft Graph from your Application

1. Register an application with Entra ID
2. Get an access token
3. Call Microsoft Graph

Registering an Application with Entra ID

Applications that authenticate with the Microsoft identity platform are registered in Microsoft Entra ID

- Registration can be created through the portal, CLI, PowerShell, or API
- Based on OAuth 2.0

Supported account types:

- Single tenant: only accounts in your organization
- Multi-tenant: accounts in any organization
- Personal Microsoft accounts: consumer accounts (e.g. Outlook.com)

Redirect URIs

- Where the identity platform is allowed to send the authentication response

Permission Requests

Permission types

- Delegated: app acts on behalf of a signed-in user
- Application: app acts as itself with no user present

Consent

- User consent: user grants delegated permissions to the app
- Admin consent: administrator grants permissions on behalf of all users in tenant
- Some permissions require admin consent regardless of who is signing in
- Permissions are embedded in the access token as scope (delegated) or roles (app) claims

Permission Requests

Dashboard > robwindstest986 | App registrations > InteractiveConsole | API permissions

InteractiveConsole | API permissions

Search (Ctrl+ /)

Refresh

Overview

Quickstart

Manage

Branding

Authentication

Certificates & secrets

Token configuration (preview)

API permissions

Expose an API

Owners

Roles and administrators (Previ...

Manifest

Support + Troubleshooting

Troubleshooting

New support request

Configured permissions

Applications are authorized to call APIs when they are granted per all the permissions the application needs. [Learn more about permi](#)

+ Add a permission

Grant admin consent for robw

API / Permissions name	Type	Descripti
Microsoft Graph (1)		
User.Read	Delegated	Sign in an

Request API permissions

All APIs

Microsoft Graph
https://graph.microsoft.com/ Docs

What type of permissions does your application require?

Delegated permissions
Your application needs to access the API as the signed-in user.

Application permissions
Your application runs as a background service or daemon without a signed-in user.

Select permissions expand all

Mail.Read ✓

Permission	Admin consent required
☒ Mail (1)	
☒ Mail.Read Read user mail ⓘ	-
☐ Mail.Read.Shared Read user and shared mail ⓘ	-
☐ Mail.ReadBasic Read user basic mail ⓘ	-
☐ Mail.ReadWrite Read and write access to user mail ⓘ	-
☐ Mail.ReadWrite.Shared	-

Add permissions

Discard

Identity APIs and SDKs

Microsoft Authentication Library (MSAL)

- General-purpose library for acquiring tokens from the Microsoft Identity Platform
- Lower-level: closer to OAuth2 and OpenID Connect protocols.

Azure Identity Client Library

- Provides TokenCredential implementations for common Entra authentication scenarios
- Can be used directly with the Microsoft Graph SDK

Microsoft Identity Web

- Recommended library for ASP.NET Core web apps and web APIs
- Wraps MSAL.NET and integrates with ASP.NET Core middleware and dependency injection
- Handles token validation in APIs automatically via `AddMicrosoftIdentityWebApi()`

Get a Delegated Access Token Using MSAL

```
private static async Task<string> GetAccessToken()
{
    var clientId = "<client id>";
    var authority = $"https://login.microsoftonline.com/{tenantName}.onmicrosoft.com/";
    var azureApp = PublicClientApplicationBuilder.Create(clientId)
        .WithAuthority(authority)
        .WithRedirectUri("http://localhost")
        .Build();

    var scopes = new string[] { "User.Read", "Mail.ReadBasic", "Files.ReadWrite" };
    var authResult = await azureApp.AcquireTokenInteractive(scopes).ExecuteAsync();

    return authResult.AccessToken;
}
```

Get an App-only (Application) Access Token Using MSAL

```
private async static Task<string> GetAccessToken()
{
    var clientId = "<client id>";
    var clientSecret = "<client secret>";
    var authority = $"https://login.microsoftonline.com/{tenantName}.onmicrosoft.com/";
    var azureApp = ConfidentialClientApplicationBuilder.Create(clientId)
        .WithAuthority(authority)
        .WithClientSecret(clientSecret)
        .Build();

    var scopes = new string[] { "https://graph.microsoft.com/.default" };
    var authResult = await azureApp.AcquireTokenForClient(scopes).ExecuteAsync();

    return authResult.AccessToken;
}
```

Call the Microsoft Graph Using HTTP – Delegated

```
private async static Task MakeCallToMicrosoftGraph()
{
    var token = await GetAccessToken();
    var client = new HttpClient();
    client.DefaultRequestHeaders.Authorization = new AuthenticationHeaderValue("Bearer", token);
    client.DefaultRequestHeaders.Accept.Add(new MediaTypeWithQualityHeaderValue("application/json"));

    var url = "https://graph.microsoft.com/v1.0/me";
    using (var response = await client.GetAsync(url))
    {
        response.EnsureSuccessStatusCode();

        var json = await response.Content.ReadAsStringAsync();
        var jsonObject = JsonObject.Parse(json);
        Console.WriteLine("Profile:");
        Console.WriteLine($"Your name is {jsonObject?["displayName"]}. Your title is {jsonObject?["jobTitle"]}");
    }
}
```

Call the Microsoft Graph Using HTTP – App-only

```
private async static Task MakeCallToMicrosoftGraph()
{
    var token = await GetAccessToken();
    var client = new HttpClient();
    client.DefaultRequestHeaders.Authorization = new AuthenticationHeaderValue("Bearer", token);
    client.DefaultRequestHeaders.Accept.Add(new MediaTypeWithQualityHeaderValue("application/json"));

    var url = $"https://graph.microsoft.com/v1.0/users('{userName}@{tenantName}.onmicrosoft.com')";
    using (var response = await client.GetAsync(url))
    {
        response.EnsureSuccessStatusCode();

        var json = await response.Content.ReadAsStringAsync();
        var jsonObject = JsonObject.Parse(json);
        Console.WriteLine("Profile:");
        Console.WriteLine($"The user's name is {jsonObject?["displayName"]}. The user's title is {jsonObject?["jobTitle"]}");
    }
}
```

Microsoft Graph SDK

Use Microsoft Graph from application code without manually handling every HTTP detail.

The SDK provides:

- Request builders for navigating Graph resources
- Strongly typed models for request bodies and responses
- Query configuration for options such as \$select, \$filter, \$orderby, and \$top
- Authentication integration
- Built-in support for common HTTP concerns such as retry handling, redirects, and payload processing

Azure Identity Credentials with the Microsoft Graph SDK

Scenario	Azure Identity credential	Access type
Desktop/console app with interactive sign-in	<code>InteractiveBrowserCredential</code>	Delegated
Device without convenient browser interaction	<code>DeviceCodeCredential</code>	Delegated
Background service using a client secret	<code>ClientSecretCredential</code>	App-only
Background service using a certificate	<code>ClientCertificateCredential</code>	App-only
Azure-hosted application	<code>ManagedIdentityCredential</code>	App-only
API calling Graph on behalf of a user	<code>OnBehalfOfCredential</code>	Delegated

GraphServiceClient uses the credential to acquire and refresh access tokens as needed.

Call the Microsoft Graph Using the Graph SDK – Delegated

```
private async static Task MakeCallToMicrosoftGraph()
{
    var options = new InteractiveBrowserCredentialOptions
    {
        TenantId = $"{tenantName}.onmicrosoft.com",
        ClientId = "<client id>",
        RedirectUri = new Uri("http://localhost")
    };

    var credential = new InteractiveBrowserCredential(options);
    var scopes = new string[] { "User.Read", "Mail.ReadBasic", "Files.ReadWrite" };
    var client = new GraphServiceClient(credential, scopes);
    var messages = await client.Me.Messages.GetAsync();
    if (messages != null && messages.Value != null)
    {
        foreach (var message in messages.Value)
        {
            Console.WriteLine($"{message?.Subject} from {message?.Sender?.EmailAddress?.Address}");
        }
    }
}
```

Call the Microsoft Graph Using the Graph SDK – App-only

```
private async static Task MakeCallToMicrosoftGraph()
{
    var credential = new ClientSecretCredential(
        tenantId: $"{tenantName}.onmicrosoft.com",
        clientId: "<client id>",
        clientSecret: "<client secret>"
    );
    var scopes = new string[] { "https://graph.microsoft.com/.default" };
    var client = new GraphServiceClient(credential, scopes);
    var userEmail = $"{userName}@{tenantName}.onmicrosoft.com";
    var messages = await client.Users[userEmail].Messages.GetAsync();
    if (messages != null && messages.Value != null)
    {
        foreach (var message in messages.Value)
        {
            Console.WriteLine($"{message?.Subject} from {message?.Sender?.EmailAddress?.Address}");
        }
    }
}
```

SharePoint Framework Microsoft Graph Client

MSGraphClientV3

- @microsoft/sp-http package
- Uses the SPFx authentication context to obtain Microsoft Graph access tokens
- Wraps the Microsoft Graph JavaScript SDK

PnPjs

- @pnp/graph package
- Alternative to MSGraphClientV3 from the M365 Platform Community
- Includes fluent API to access Microsoft Graph

Can optionally use with Microsoft Graph Type Declarations

- @microsoft/microsoft-graph-types package
- Work with strongly-typed objects representing Graph data

SPFx Microsoft Graph Client Code

```
import { MSGraphClientV3 } from '@microsoft/sp-http';
import { GraphError } from '@microsoft/microsoft-graph-client';
import * as MicrosoftGraph from '@microsoft/microsoft-graph-types';

-----

export interface IGroupCollection {
  value: MicrosoftGraph.Group[];
}

-----

this.context.msGraphClientFactory
  .getClient('3')
  .then((client: MSGraphClientV3): void => {
    client.api('/groups')
      .orderBy('displayName')
      .get((error: GraphError, response: IGroupCollection) => {
        const elem: Element = this.domElement.querySelector("#groupsContainer");
        elem.innerHTML = response.value.map((group) => {
          return `<span>${group.displayName}</span>`;
        }).join(`<br />`);
      });
  });
```

SPFx Solutions Declare Permission Requests

When using MSGraphClientV3 you do not need to register an Entra ID app

- SPFx uses a Microsoft-managed Entra application/service principal for Graph access

Your solution declares the permissions it requires

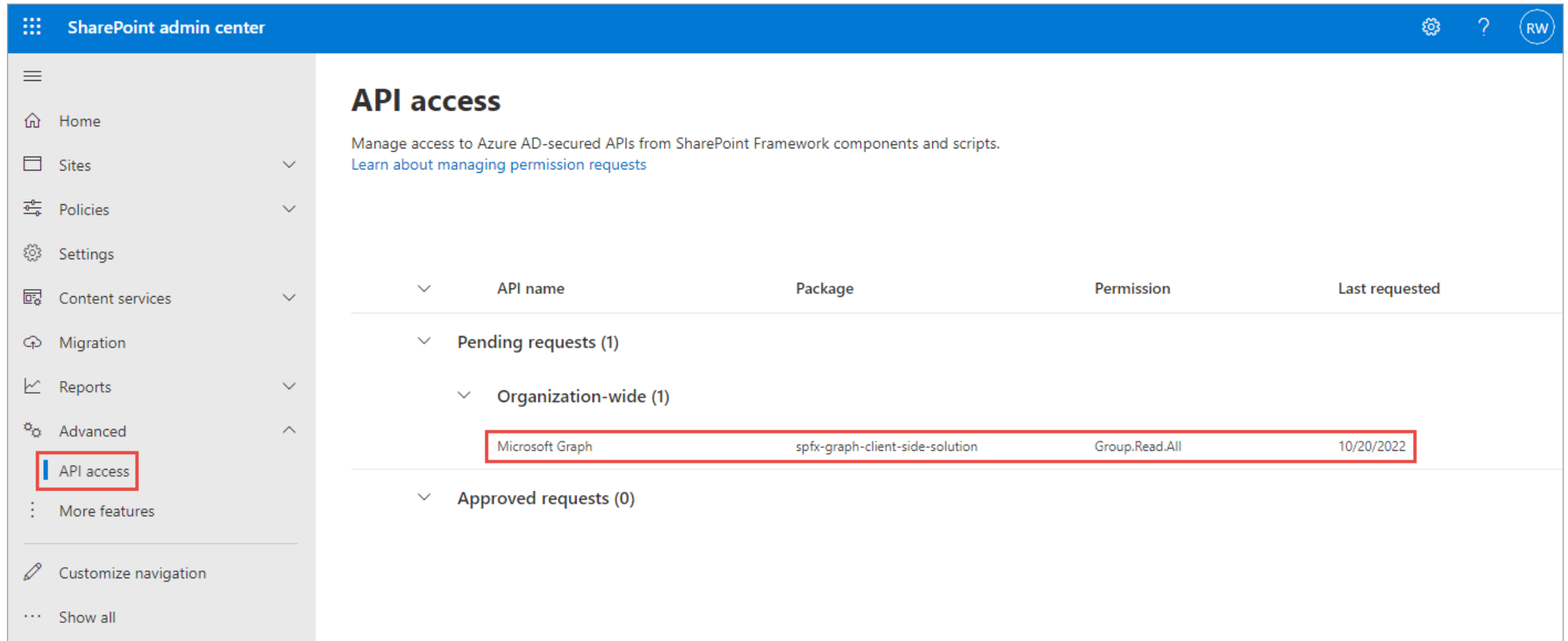
Tenant administrator grants or rejects permission requests using an API Access page in the SharePoint Online administration portal

- Granting the permission request updates the Entra ID app created by Microsoft

SPFx Solutions Declare Permission Requests

```
{
  "$schema": "https://developer.microsoft.com/json-schemas/spfx-build/package-solution.schema.json",
  "solution": {
    "name": "MSGraphClient Web Part",
    "id": "b9cbefa9-8399-42a3-923b-5261be6abf23",
    "version": "1.0.0.0",
    "includeClientSideAssets": true,
    "skipFeatureDeployment": true,
    "isDomainIsolated": false,
    "webApiPermissionRequests": [
      {
        "resource": "Microsoft Graph",
        "scope": "Group.Read.All"
      }
    ]
  },
  "paths": {
    "zippedPackage": "solution/ms-graph-client.sppkg"
  }
}
```

Approve / Reject API Access Requests



The screenshot shows the SharePoint admin center interface. The left-hand navigation pane includes links for Home, Sites, Policies, Settings, Content services, Migration, Reports, Advanced, API access (highlighted with a red box), and More features. The main content area is titled "API access" and includes a description: "Manage access to Azure AD-secured APIs from SharePoint Framework components and scripts." Below this, there is a table with columns for API name, Package, Permission, and Last requested. The table shows one pending request for "Microsoft Graph" with the package "spfx-graph-client-side-solution" and permission "Group.Read.All", requested on "10/20/2022". The request is highlighted with a red box. There are also sections for "Pending requests (1)" and "Approved requests (0)".

SharePoint admin center

API access

Manage access to Azure AD-secured APIs from SharePoint Framework components and scripts.
[Learn about managing permission requests](#)

API name	Package	Permission	Last requested
Pending requests (1)			
Organization-wide (1)			
Microsoft Graph	spfx-graph-client-side-solution	Group.Read.All	10/20/2022
Approved requests (0)			

Approve / Reject API Access Requests

×

Approve access

If you approve access, any SharePoint Framework component or custom script can call this Azure AD-secured API with "Group.Read.All" permission.

API name Microsoft Graph	Package name spfx-graph-client-side-solution
Permission Group.Read.All	Version 1.0.0.0
Requested by Rob Windsor	Last requested 10/20/2022

Approve **Cancel**

Tools and Resources

Microsoft Graph Developer Center

- graph.microsoft.com

Microsoft Graph Documentation

- learn.microsoft.com/en-us/graph

Microsoft Graph Tutorials

- learn.microsoft.com/en-us/graph/tutorials

Microsoft Graph Explorer

- developer.microsoft.com/en-us/graph/graph-explorer

Microsoft 365 Developer Program

- developer.microsoft.com/en-us/microsoft-365/dev-program

Microsoft Learn Training

Microsoft Graph Fundamentals

- learn.microsoft.com/en-us/training/paths/m365-msgraph-fundamentals/

Explore Microsoft Graph scenarios for ASP.NET Core development

- learn.microsoft.com/en-us/training/paths/m365-msgraph-dotnet-core-scenarios/

Explore Microsoft Graph scenarios for JavaScript development

- learn.microsoft.com/en-us/training/paths/m365-msgraph-scenarios/

SPFx - Use Microsoft Graph and non-Microsoft APIs

- learn.microsoft.com/en-us/training/modules/sharepoint-spfx-graph-3rd-party-apis/